EnergyPilot: A Proactive Agentic Home Energy Optimizer

Emil Ljungdell
Group EnergyPilot
Hanyang University

Philip Dahlensjö
Group EnergyPilot
Hanyang University

Filip Kadmark
Group EnergyPilot
Hanyang University

Jasmin Hansson
Group EnergyPilot
Hanyang University

TABLE I
ROLE ASSIGNMENTS (TILL THE MIDDLE OF THE TERM)

Roles	Name	Task description
User	Philip Dahlensjö	Sets intents and preferences, tries the prototype and reports usability problems.
Customer	Emil Ljungdell	Represents the household that pays the bill. Prioritizes the requirements and accepts the result.
Software developer	Jasmin Hansson	Designs and implements the agent, the scheduler, the backend and the user interface.
Development manager	Filip Kadmark	Tracks deadlines, leads group meetings, follows up tasks and makes sure each deliverable is submitted.

***Abstract*—EnergyPilot is a proposal for a software agent that runs household appliances at the cheapest suitable time, without the resident having to program anything. The resident states an intent, for example "the laundry should be done by 07:00, and keep the cost low", and the agent turns it into a schedule based on forecast electricity prices and household usage. It acts on its own or asks for approval, explains every decision, and plans again when prices or devices change. This document extends our requirement document with the specification phase. It lists eleven requirements in one list, describes the development environment (a Python and TypeScript web application in Linux containers, with an estimated cost of at most 44 USD), reviews the software and algorithms we build on, and specifies at pseudocode level how each requirement will be implemented. The first version uses simulated appliances and synthetic or public data behind a device interface that can later be replaced by real devices.**

***Index Terms*—smart home, agentic AI, energy optimization, requirements, specification, scheduling**

I. INTRODUCTION

A. Motivation

Electricity prices change during the day, but most households run their appliances when it suits them, which is often when power is expensive. Moving flexible loads such as laundry and dishwashing to cheaper hours lowers the bill and can also ease the peak load on the grid. Doing this by hand takes effort: you need to know the prices, remember to start the machine and be at home when it is time. We think an agent can do this for the resident. It also matches this year's course topic, a proactive agentic home that predicts, decides and acts before a problem occurs.

B. Problem Statement and Client Needs

The client is a household that pays the electricity bill and owns several connected appliances. Its needs are:

- lower electricity cost without losing convenience (the laundry must still be done when the resident needs it),
- no manual programming of schedules or rules,
- full control, so the resident can always override the agent,
- understandable decisions, so the resident can see why the agent did something, and
- careful handling of usage data, since it shows when people are home.

C. Related Software

LG ThinQ lets users control LG appliances remotely, build smart routines and monitor energy use, including a comparison with neighbors and savings goals with notifications [1]. The routines are built by the user, and the energy features mainly report what has already happened.

Samsung SmartThings Energy has an AI Energy Mode that adjusts supported Samsung appliances and TVs to use less energy on selected cycles, and an Away mode for turning devices off when nobody is home [2], [3]. The optimization mostly happens inside each device.

Google Home shows energy history and an energy dashboard for Nest thermostats, with separate heating and cooling totals, and the Nest Learning Thermostat builds a schedule from the household's routine [4], [5]. The energy functions we found are centered on the thermostat.

Matter is a connectivity standard that lets devices from different brands work together [6]. It is relevant if EnergyPilot is later connected to real devices.

In the material we found, none of these products lets the user state an intent (a deadline and a cost preference) that an agent turns into a schedule across several appliances based on electricity prices. We will verify this in the apps before the final report.

D. Scope

The first version covers scheduling of flexible appliances (washing machine, dryer, dishwasher and one climate device), price and usage forecasting, a user interface for intents and overrides, and a savings report. Real hardware, billing integration and solar production are out of scope for now.

II. Requirements

The requirements are given in one list. Each is prioritized as Must or Should and can change during the project. Section IV specifies how each requirement is achieved, in the same order.

A. R1: Appliance registration and device interface (Must)

The user registers an appliance with its type, power in watts, typical cycle length and earliest start. All appliances are accessed through one device interface, so simulated appliances can later be replaced by real ones without changing the rest of the system.

B. R2: Intent input (Must)

The user states a goal as free text, such as "done by 07:00" or "cheapest possible today", or with a simple form. The system shows how it understood the goal and asks for confirmation. A new user can register an appliance and set an intent in under five minutes.

C. R3: Electricity prices (Must)

The system gets hourly prices for the next 24 to 48 hours from a price source behind one interface. The first source is a synthetic time-of-use tariff, and public price data can be added through the same interface.

D. R4: Price and usage forecasting (Should)

The system predicts hourly prices and household consumption for the coming day. It compares several forecasting methods on past data and uses the one with the lowest error.

E. R5: Schedule generation (Must)

For each intent the agent chooses the start time with the lowest cost that respects the deadline, the quiet hours and the maximum household load. A schedule for the next 24 hours is ready in under 10 seconds, and a plan that breaks a hard constraint is never executed.

F. R6: Act or ask (Should)

The user chooses, per appliance, whether the agent starts it automatically or asks for approval first.

G. R7: Decision log and explanations (Must)

Every decision is saved with a short explanation in plain language, for example "Started the washer at 02:00, 38 % cheaper than 18:00". The numbers in an explanation always come from the scheduler.

H. R8: Savings dashboard (Should)

A dashboard shows consumption, cost and estimated savings compared with a baseline without the agent.

I. R9: Manual override (Must)

The user can change, cancel or start a planned run at any time. Manual control always wins over the agent.

TABLE II
Development environment

Item	Choice
Developer machines	**[TODO: each member adds OS version, CPU and RAM]**
Server OS	Ubuntu Linux LTS in Docker containers
Backend	Python 3.12, FastAPI, Pydantic v2
Database	PostgreSQL 16 (SQLite in unit tests), SQLAlchemy 2
Data and ML	pandas, NumPy, scikit-learn
Frontend	TypeScript 5, React, Vite
Containers	Docker and Docker Compose
Tests and CI	pytest, GitHub Actions
LLM	LLM API with tool calling, used only to read the intent and to word explanations
Documents	LaTeX (IEEEtran), Overleaf

J. R10: Replanning (Should)

If prices change or an appliance goes offline, the agent makes a new plan for the runs that have not started and tells the user.

K. R11: Security and privacy (Must)

Login is required, communication is encrypted, and only the data needed for scheduling is stored. The user can delete all of their own data.

III. Development Environment

A. Choice of Software Development Platform

Platform. EnergyPilot is a web application. The backend runs in Linux containers (Docker), and the user interface runs in any modern browser on a phone or a computer. We chose the web because the user installs nothing, the same build works on every operating system, and a demo only needs a browser. We rejected a native mobile app, because builds and test devices would take time that we need for the agent itself.

Programming languages. The backend, the forecasting and the scheduler are written in Python 3.12. We need its data libraries (pandas, NumPy, scikit-learn), which we also use in the AI course, and FastAPI gives us a typed REST API that generates its own documentation. The frontend uses TypeScript with React, because type checking catches mistakes in the data we get from the API.

Version control and documents. The code and this document are in a public GitHub repository, https://github.com/PhilipDahlensjo/SE-AI_project. The documents are LaTeX files in the repository (IEEEtran, also usable in Overleaf). We track work with issues and pull requests, and each issue names the requirement it implements.

Development environment. Table II lists what we use. Exact library versions are fixed in the lock files in the repository.

Cloud (bonus). We plan to deploy on AWS Lightsail with the Small-2GB Linux plan (2 vCPU, 2 GB RAM, 60 GB storage, 12 USD per month), running the backend, the database and the frontend with Docker Compose [14]. Developers also run the same Compose file locally.

TABLE III
COST ESTIMATION

Item	USD
Open-source software, GitHub, Overleaf (free plans)	0
Developer computers (already owned)	0
AWS Lightsail Small-2GB, about 2 months	24
LLM API usage (spending limit)	≤ 20
Total	$\leq$ **44**

Cost estimation. Table III shows the estimate. All software is free or open source, and we need no new hardware. The LLM cost is a limit, not a price: we set a spending limit of 20 USD in the provider's console, and the usage (a few hundred short requests per week) should stay far below it.

B. Software in Use

Products that do a similar task are described in Section I-C. None of them schedules several appliances from a stated intent. We build on the following existing software and algorithms:

- *Seasonal naive forecast*, which predicts the value from 24 hours earlier [8], as the baseline forecaster.
- *Ridge regression* and *gradient boosting* [9] from scikit-learn [10] for price and consumption forecasts.
- *Exhaustive search* over start slots in the scheduler. If a household load limit makes several appliances depend on each other, the CP-SAT solver in OR-Tools [11] is the next step.
- *FastAPI* [12] for the REST API.
- *LLM tool calling* [13] to turn the resident's text into structured data.
- Later, optionally, the *LG ThinQ Connect API* [7] and *Matter* [6] for real appliances.

C. Task Distribution

As the assignment allows, the task distribution will be given in the design document (C). The roles until then are in Table I.

IV. SPECIFICATIONS

Fig. 1 shows how the parts fit together, and Table IV lists the main data. Time is divided into 15-minute slots ($\Delta t = 0.25$ h), so 48 hours are 192 slots. Each subsection below belongs to the requirement with the same number in Section II.

A. R1: Appliance registration and device interface

A device is stored with its data, and every action goes through an adapter that has the same three operations for simulated and real appliances. The simulated adapter runs a cycle on the simulator clock.

procedure REGISTERDEVICE(u, $spec$)
  check $spec.power > 0$ and $spec.cycle \geq 15$ min
  $d \leftarrow$ SAVE(Device(u, $spec$))
  $d.adapter \leftarrow$ ADAPTERFOR($spec.adapter$) ▷ simulated by default
  return d
end procedure

interface DeviceAdapter: STATE(), START(), STOP()

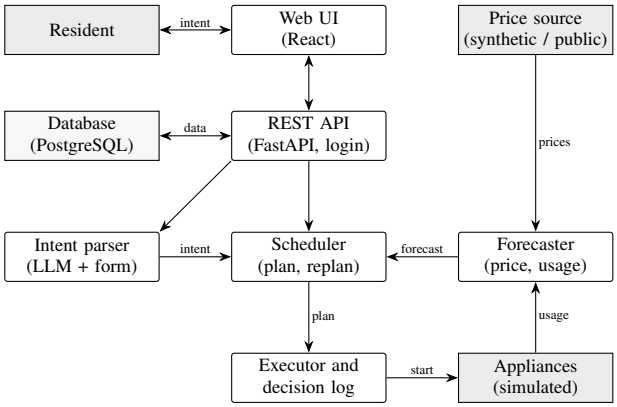


Fig. 1. System overview: main parts and data flows.

TABLE IV
MAIN DATA

Entity	Main fields
User	id, e-mail, password hash
Device	id, user, type, power (W), cycle (min), mode (auto/ask), adapter
Intent	id, device, earliest start, deadline, goal (cheapest/fastest), status
Price	slot start, price per kWh (relative unit in synthetic data)
Consumption	slot start, kWh, source
Run	id, device, intent, start, cost, baseline cost, status, manual flag
DecisionLog	time, run, action, explanation text

B. R2: Intent input

The LLM only converts text to a fixed JSON format (device, deadline, goal). The result is checked against a schema, and a plain form is the fallback, so the demo also works without the LLM.

procedure SETINTENT(u, $text$)
  $j \leftarrow$ LLMPARSE($text$) ▷ JSON: device, deadline, goal
  if LLM unavailable **or** j does not match schema **then**
    $j \leftarrow$ ASKFORM(u) ▷ fallback
  end if
  show j in plain words
  if user does not confirm **then**
    return
  end if
  $i \leftarrow$ SAVE(Intent(u, j, pending))
  PLAN(i)
end procedure

C. R3: Electricity prices

Prices are read from the database first and fetched from the price source only if slots are missing. The synthetic tariff has three price levels over the day plus noise.

```
procedure GetPrices(t_0, H)                                   ▷ H = 48 h
    p ← Load(prices in [t_0, t_0 + H))
    if p is complete then
        return p
    end if
    p ← source.Fetch(t_0, H)
    for each slot t with p_t missing or p_t < 0 do
        p_t ← p_{t−1}                                         ▷ last known value
    end for
    Save(p); return p
end procedure

procedure SyntheticTariff(t)
    k ← level of hour(t) in {off-peak, mid, peak}
    return base[k] · (1 + ε), ε ∼ N(0, σ^2)
end procedure
```

D. R4: Price and usage forecasting

We compare three methods with the last seven days as validation data and use the one with the lowest mean absolute error. This component is shared with our project in the AI course, which uses public price and consumption data (see the repository).

```
procedure Forecast(hist)                                      ▷ next 24 h
    train ← hist without last 7 days; val ← last 7 days
    for m ∈ {SeasonalNaive, Ridge, GradientBoosting} do
        fit m on train; features: hour, weekday, y_{t−24}, y_{t−168}
        err[m] ← MAE(m.predict(val), val)
    end for
    m* ← arg min_m err[m]; refit m* on all of hist
    return m*.predict(next 24 h), err
end procedure
```

E. R5: Schedule generation

For one appliance the agent tries every allowed start slot and keeps the cheapest. The cost of a start slot s for an appliance with power P and a cycle of L slots, using price c_t, is

$$C(s) = \sum_{t=s}^{s+L-1} c_t \, P \, \Delta t. \tag{1}$$

With 192 slots the search is far below 10 seconds. With several intents, the one with the least slack ($deadline - earliest - L$) is planned first, and each result is added to the load l_t.

```
procedure Plan(i)
    d ← i.device; L ← ⌈d.cycle/15⌉; P ← d.power
    c ← GetPrices(now, 48 h); l ← load of other planned runs
    best ← ⊥; bestCost ← ∞
    for s ← i.earliest to i.deadline − L do
        if slots s..s + L − 1 touch quiet hours then
            continue
        end if
        if ∃t : l_t + P > L_max then
            continue
        end if
        C ← Σ_{t=s}^{s+L−1} c_t P Δt
        if C < bestCost then
            best ← s; bestCost ← C
        end if
    end for
    if best = ⊥ then
        mark i infeasible; tell the user; return
    end if
    assert HardRulesOK(best)
    return Save(Run(d, best, bestCost))
end procedure
```

F. R6: Act or ask

A dispatcher runs every minute. A run is only executed after the hard rules are checked again, because prices or the state of the appliance may have changed.

```
procedure Dispatch                                            ▷ every minute
    for each planned run r with r.start − now ≤ δ do
        if r.device.mode = auto then
            Execute(r)
        else
            AskUser(r)                                        ▷ once
            if approved then
                Execute(r)
            else if declined or no answer at r.start then
                cancel r; Log(r, "not started")
            end if
        end if
    end for
end procedure

procedure Execute(r)
    if HardRulesOK(r) and adapter.state = idle then
        adapter.start(); r.status ← running; Log(r)
    else
        Replan(r)
    end if
end procedure
```

G. R7: Decision log and explanations

The explanation is first made from a template with numbers from the scheduler. The LLM may reword it, but the new text is only used if it contains no number that is missing in the original.

```
procedure Explain(r)
    base ← cost if started at the earliest allowed time
    pct ← 100 (base − r.cost)/base
    text ← Template(r.device, r.start, pct)
    if LLM available then
        t' ← LLMReword(text)
        if every number in t' is in text then
            text ← t'
        end if
    end if
    Save(DecisionLog(now, r, text))
end procedure
```

H. R8: Savings dashboard

The baseline is a resident without the agent, who starts each run at the earliest time the intent allows. The saving is

$$S = 100 \, \frac{C_{base} - C_{agent}}{C_{base}} \ \%. \tag{2}$$

```
procedure Report(u, from, to)
    R ← finished runs of u in [from, to]
    C_agent ← Σ_{r∈R} r.cost; C_base ← Σ_{r∈R} r.baseCost
    S ← 100 (C_base − C_agent)/C_base
    return daily series (cost, baseline, kWh) and S
end procedure
```

I. R9: Manual override

A run that is already going can only be stopped, because a washing cycle cannot be moved. After any change the agent plans the other runs again, since the load has changed.

```
procedure OVERRIDE(r, action)                    ▷ cancel, move(t), start now
  if r is running and action ≠ cancel then
    return error
  end if
  apply action through the adapter; r.manual ← true
  LOG(r, "changed by user")
  REPLAN(other planned runs)
end procedure
```

J. R10: Replanning

Planning is triggered by events. Runs that are manual or already running are never touched.

```
procedure ONEVENT(e) ▷ price update, device offline, deadline change
  A ← planned runs affected by e, sorted by slack
  for r ∈ A with not r.manual do
    r′ ← PLAN(r.intent)                          ▷ current prices
    if no plan found then
      mark r infeasible; notify the user
    else if r′.start ≠ r.start then
      update r; notify the user; LOG(r, e)
    end if
  end for
end procedure
```

K. R11: Security and privacy

Passwords are stored as Argon2id hashes, requests carry a short-lived token, and TLS is terminated at a reverse proxy with a free certificate. Only 15-minute consumption values are kept.

```
procedure LOGIN(email, pw)
  if Argon2id.verify(user.hash, pw) then
    return token(user, expires in 1 h)
  end if
  return error
end procedure

procedure HANDLE(request)
  verify token
  if resource.owner ≠ token.user then
    return 403
  end if
end procedure

procedure DELETEMYDATA(u)
  delete devices, intents, runs, consumption and log of u
end procedure
```

REFERENCES

[1] LG Electronics, "LG ThinQ" (app description). Accessed: Nov. 5, 2026. [Online]. Available: https://www.apkmirror.com/apk/lgelectronics/lg-smartthinq

[2] SmartThings, "New SmartThings Energy features take home energy management to next level," Samsung Newsroom, Jun. 27, 2023. Accessed: Nov. 5, 2026. [Online]. Available: https://news.samsung.com/us/samsung-new-smartthings-energy-features-take-home-energy-management-next-level

[3] Samsung, "How to use SmartThings AI energy saving features." Accessed: Nov. 5, 2026. [Online]. Available: https://www.samsung.com/ie/support/apps-services/how-to-use-smartthings-ai-energy-saving-controls/

[4] Google, "Nest Thermostat," Google Store. Accessed: Nov. 5, 2026. [Online]. Available: https://store.google.com/product/nest_thermostat?hl=en-us

[5] "Google Home app update brings new features for smart thermostats," Notebookcheck. Accessed: Nov. 5, 2026. [Online]. Available: https://www.notebookcheck.net/Google-Home-App-update-brings-new-features-for-smart-thermostats.1098863.0.html

[6] Connectivity Standards Alliance, "Matter." Accessed: Nov. 5, 2026. [Online]. Available: https://csa-iot.org/all-solutions/matter/

[7] LG Electronics, "ThinQ Connect: Personal Access Token (PAT)." Accessed: Nov. 5, 2026. [Online]. Available: https://thinq.developer.lge.com/en/cloud/docs/thinq-connect/PAT-en

[8] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: https://otexts.com/fpp3/

[9] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001.

[10] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[11] L. Perron and V. Furnon, "OR-Tools," Google. Accessed: Nov. 5, 2026. [Online]. Available: https://developers.google.com/optimization/

[12] S. Ramírez, "FastAPI." Accessed: Nov. 5, 2026. [Online]. Available: https://fastapi.tiangolo.com/

[13] Anthropic, "Claude API documentation." Accessed: Nov. 5, 2026. [Online]. Available: https://docs.claude.com

[14] Amazon Web Services, "Lightsail instance bundles." Accessed: Nov. 5, 2026. [Online]. Available: https://docs.aws.amazon.com/lightsail/latest/userguide/amazon-lightsail-bundles.html